Selecting Reliable LLM Reasoning with Embeddings and Candidate Agreement

DASE7506 Advanced Machine Learning Methods
Mini Project 2 Research Report

Zhongtian Zhang
Student ID 3036707587
29 October 2026

Abstract

I study how to select a correct final numeric answer from four supplied language-model solutions. The retained V2 method combines fixed candidate embeddings, within-question differences, answer agreement and observable text features in a small shared multilayer perceptron. A groupwise objective assigns probability to every correct candidate rather than selecting one arbitrary training target. V2 achieves 736/798 correct selections on validation (92.23%) and 374/467 on the public test bank (80.09%). Majority voting reaches 86.22% on validation. Removing embedding information reduces validation accuracy to 88.47%, whereas removing the explicit vote prior gives 92.11%. Error analysis shows substantial gains on disagreements but persistent difficulty when only one candidate is correct. Later arithmetic, context and pairwise experiments do not improve the observed public test result. Because those experiments followed an initial test evaluation, the reported test comparison is descriptive and is not an untouched generalization estimate.

1 Task and research questions

The course task is offline selection: given a question, four candidate traces and fixed 1024-dimensional representations, return one candidate index from 0 to 3 [1, 2]. Every question has at least one correct candidate. The official parser and scorer compare final numeric answers, including equivalent fractions and decimals; a correct selection does not establish that every intermediate reasoning step is valid.

Split	Questions	Candidate traces	Role
Training	4,755	19,020	Fitting parameters and statistics
Validation	798	3,192	Development and selection
Test	467	1,868	Public scoring and reproduction

The main research question is whether learned candidate representations help beyond final-answer agreement. I also examine whether the explicit vote prior contributes when agreement is already an input, and whether improvements remain when evaluation excludes easy agreement groups. The expected benefit is better discrimination when otherwise plausible candidates disagree.

All methods use supplied inputs only. No external dataset, additional pretrained weights, generation or API calls are used for prediction. IDs serve alignment only. The official data, numeric parser and scoring code are unchanged.

2 Selection method

2.1 Initial baselines

Majority voting parses the four final answers with common.parse_answer, chooses the most frequent valid value and returns its earliest candidate. Ties use the earliest candidate; if every parse fails, the baseline returns index 0. The first learned baseline is logistic regression over unit-normalized 1024-dimensional embeddings, supervised by whether each candidate matches the training reference. Multiple positive candidates are retained. Inference selects the highest score.

2.2 V2 features and candidate scorer

For candidate j, e_j is its unit-length embedding and e_bar is the mean of the four unit embeddings in the current question. V2 forms the following 2071-dimensional vector. It does not separately encode the question text.

$$\mathrm{x_j} = \mathrm{concatenate}(\mathrm{e_j},\ \mathrm{e_j} - \mathrm{e_bar},\ \mathrm{a_j})$$

Feature family	Dimensions	Purpose
Unit candidate embedding	1,024	Trace representation
Difference from question mean	1,024	Relative candidate information
Auxiliary features a_j	23	Agreement and observable evidence
Total	2,071	Shared scorer input

The 23 auxiliary values contain parseability; answer vote count, maximum vote count, distinct valid-answer count and invalid count; log character count, word count, line breaks, equals signs and digit groups; answer-format markers; selected revision and connective words; three mean embedding-similarity statistics; log original embedding norm; and five within-question relative text statistics. Similarities summarize other candidates with the same answer, candidates with different valid answers and all other candidates.

Feature means and standard deviations are fitted only on training inputs. Standard deviations have a floor of 0.05; standardized values are clipped to [−8, 8]. A shared 64-unit ReLU layer and scalar linear output give s_j. Sharing weights across candidates avoids a separate learned scoring rule for each candidate position. Dropout is applied only during training.

2.3 Combining the learned score with agreement

$$\mathrm{z_j} = \mathrm{s_j} + \alpha\ \log(\max(\mathrm{v_j},\ 0.1)) \qquad \mathrm{choice} = \mathrm{argmax_j}\ \mathrm{z_j}$$

Here v_j counts candidates sharing the same valid parsed final answer; invalid parses have zero votes before smoothing. The selected V2 inference coefficient is α = 1. Agreement is also present in the auxiliary inputs. Relative features and the prior make selection depend on the whole question group, although the final network evaluates candidates with shared weights.

2.4 Groupwise training objective

$$\mathrm{p_j} = \exp(\mathrm{z_j})\ /\ \Sigma_\mathrm{k}\ \exp(\mathrm{z_k}) \qquad \mathrm{L} = -\log(\Sigma_\mathrm{j}\ \mathrm{y_j}\ \mathrm{p_j})$$

The training label y_j equals 1 when the parsed candidate answer matches the training reference. The loss rewards total probability on any correct candidate. An all-correct question has zero ranking gradient; weight regularization can still act. This objective is aligned with the evaluator accepting any correct candidate and differs from forcing one arbitrary correct index.

3 Experimental protocol and validation results

The original V2 study fits weights and preprocessing on 4755 training questions. Validation chooses the checkpoint and inference vote coefficient. It is also used for error inspection, so validation accuracy is a selected development result. Adam optimizes the group loss with L2 weight regularization; the complete method uses a training vote coefficient of 1.

Setting	Logistic baseline	V2 and main ablations
Seed	17	17
Epoch budget	200	60
Learning rate	0.01	0.001
L2 weight coefficient	0.001	0.001
Hidden units	None	64
Batch size	Full batch	128 questions
Dropout	None	0.15
Validation checks	Every epoch	Epoch 1 and every 5 epochs
Selected epoch	26	5

V2 searches α in {0, 0.5, 1, 2, 4} at scheduled validation checks. Only a strictly higher correct count replaces the checkpoint; ties retain the earlier choice. The 60-epoch run is completed rather than stopped at epoch 5. A majority-vote fallback is available but is not selected.

Method	Correct / 798	Accuracy	Change vs V2
Majority voting	688	86.22%	−6.02 pp
Embedding logistic regression	667	83.58%	−8.65 pp
Complete V2	736	92.23%	Reference
V2 without embedding information	706	88.47%	−3.76 pp
V2 without explicit vote prior	735	92.11%	−0.13 pp

3.1 What the ablations show

The no-embeddings ablation zeros the 2048 direct embedding coordinates, three similarity values and original norm feature before fitting statistics. It keeps the original feature width and 64-unit network, so it tests information removal rather than computational savings. It is retrained and selects $\alpha = 2$. Losing 30 correct selections supports a useful contribution from embedding information under this protocol.

The no-explicit-prior ablation removes the additional log-vote term during both training and inference; α is fixed to 0. It retains vote counts and other agreement inputs. Its one-question deficit is weak evidence of an additional prior benefit and does not show that agreement itself is unnecessary. Both ablations use the same seed and budget and do not use majority fallback.

Logistic regression is worse than majority voting. This does not imply that embeddings are uninformative: V2 changes the features, scorer and objective together. Neither this comparison nor a single-seed ablation establishes a causal account of what the network learned or statistically significant superiority.

4 Error analysis and evaluation difficulty

Validation relationship to majority voting	Questions
Both correct	669
Majority wrong and V2 correct	67
Majority correct and V2 wrong	19
Both wrong	43

V2 corrects 67 majority errors while introducing 19, for a net gain of 48. Errors fall from 110 to 62, a 43.64% descriptive relative reduction on validation. Of the 62 V2 errors, 7 reject a unique correct plurality, 28 occur when the unique plurality is wrong, and 27 occur on plurality ties. These are answer-pattern descriptors, not verified causes such as arithmetic mistakes or misunderstood conditions.

Validation subset	N	Majority correct	V2 correct
At most one distinct valid answer	443	443 (100.00%)	443 (100.00%)
At least two distinct valid answers	355	245 (69.01%)	293 (82.54%)
All four valid answers agree	379	379 (100.00%)	379 (100.00%)
Exactly one correct candidate	117	27 (23.08%)	69 (58.97%)
Two pairs of valid answers	34	14 (41.18%)	27 (79.41%)
Four distinct valid answers	20	5 (25.00%)	13 (65.00%)

The first two rows partition validation; other rows overlap them and must not be summed. Forty-eight of the 62 remaining errors occur when exactly one candidate is correct, accounting for 77.42% of V2 errors. This subset remains difficult even though V2 improves substantially over majority voting.

4.1 Why the complete validation score is higher than test

The guide specifies that all test groups contain at least two distinct valid parsed answers, whereas training and validation include easier groups [1, 2]. The full validation score decomposes as (443 + 293)/798 = 92.23%. Comparing the 82.54% disagreement-only validation result with 80.09% on test is more informative than comparing 92.23% directly with 80.09%. The remaining gap is about 2.45 percentage points. This composition difference explains part of the apparent gap, but does not prove that the difficult subsets have identical distributions or explain every error.

4.2 A corrected case

A validation question describes an 80-minute concert, including a 10-minute intermission, one 10-minute song, and other songs lasting 5 minutes. The four final answers are 16, 14, 13 and 12. Majority voting ties and selects 16; V2 selects the correct answer 13. There are 70 minutes of singing: one long song plus twelve short songs. The successful selection does not reveal whether the network explicitly verified this calculation.

The error-analysis exports retain all candidates, parsed answers, reference answers and selections, including corrected, introduced-error and common-error groups. Qualitative claims about reasoning should be based on reading those traces, not inferred solely from automated categories.

5 Subsequent experiments and public test results

After the first V2 test evaluation, I investigated context embeddings, arithmetic consistency and pairwise ranking. These are subsequent development experiments. They were trained on supplied training data, with validation or inner training splits used for checkpoint selection; public test results were also viewed across versions. The final decision to retain V2 followed these comparisons. This departs from a strict freeze-before-first-test model-selection protocol, even though the later fitting procedures did not use test labels. The public score is reproducible, but should not be presented as an untouched estimate of generalization; eligibility remains subject to course review.

Evaluated model	Validation correct / 798	Test correct / 467	Test accuracy
Retained original V2	736	374	80.09%
V3 representation ensemble	740	372	79.66%
V2 with arithmetic features	740	373	79.87%
V2 features with pairwise loss	739	364	77.94%

The retained result was reproduced after directory cleanup: 374/467, with official accuracy 80.08565310492506. Repeating evaluation of the same checkpoint confirms reproducibility; it is not a new independent experiment. The small differences among V2, V3 and arithmetic versions do not support a broad claim of architectural superiority.

5.1 Training-only cross validation

The arithmetic comparison uses nested three-fold splits of the 4755-question training bank. In each fold, the 3170 outer-fit questions are split into 2853 fitting and 317 selection questions. Epoch and vote coefficient are chosen internally, then the model is retrained from scratch on all 3170 questions and evaluated on the 1585 outer holdout questions. No official validation or test is read within that run. The folds are disjoint, although repeated development on the dataset still limits independence.

Windows arithmetic CV	All correct / 4755	Hard correct / 2114	Single correct / 711
Paired V2 baseline	4296 (90.35%)	1655 (78.29%)	374 (52.60%)
Arithmetic features	4308 (90.60%)	1667 (78.86%)	417 (58.65%)

Arithmetic features net 43 extra selections in single-correct groups, but only 12 overall; the remaining groups therefore lose 31 net selections. The full validation run reaches 740/798, yet its public test result is 373/467. Better development scores did not establish better test performance.

5.2 Embedding context and pairwise ranking

A separate Linux training-only nested comparison uses the same three-fold design and 60-epoch limit. The paired baseline scores 4298/4755, the original-feature pairwise model 4321/4755, expanded context embeddings 4286/4755, and expanded context with pairwise loss 4279/4755. Pairwise net gains by fold are 8, 15 and 0 questions, but this gain does not reproduce on the public test bank. The loss averages $\log(1 + \exp(s_{wrong} - s_{correct}))$ over correct-versus-incorrect pairs per question, ignoring equal-label pairs. These runs are not pooled with Windows results because the numerical environment differs.

Other text-ranking, score-stacking, answer-group and question-number compatibility experiments were unsuccessful or inconsistent in training-only comparisons. They are retained as negative development records, not as evidence of a stronger final method.

6 Computational resources and checkpoint identity

Student-side evaluation runs on Windows 11, Python 3.12.10 and NumPy 2.5.3 with an AMD Ryzen 9 7940HX with Radeon Graphics. OMP_NUM_THREADS, OPENBLAS_NUM_THREADS and MKL_NUM_THREADS are set to 4 before Python starts. Prediction is CPU-only and requires no model download or online request. Assistant-side reference experiments use Linux, Python 3.12.14 and NumPy 2.3.5; their timing is reported separately.

Recorded run	Training or search time	Evaluation measurement
Logistic regression on Windows	67.67 s	Full process time not recorded
V2 no-embeddings on Windows	15.64 s	Validation fresh processes 1.282 s
V2 no-prior on Windows	15.27 s	Validation fresh processes 1.273 s
Arithmetic nested CV on Windows	75.49 s total	Training bank only
Embedding and pairwise CV on Linux	112.11 s total	Four methods across three folds
Original V2 reference run on Linux	9.50 s	736/798 validation

A separate exact Windows training duration for the retained original V2 is not available in the supplied run transcript; the Linux duration is not substituted for it. These are recorded run costs, not a complete cumulative account of every exploratory experiment.

6.1 Test timing and eligibility limits

The latest V2 test reproduction reports 0.3803545 s inside prediction and 0.0544615 s inside scoring, totaling approximately 0.435 s. These fields exclude part of process startup and therefore do not satisfy the course end-to-end timing definition by themselves. An earlier student-side external stopwatch recorded 2.8158724 s; interactive command entry may contribute overhead, so it is an approximate local observation. The required reference-machine check is one fresh prediction process plus the independent scorer on the Xeon Platinum 8457C, with four threads total and a 30 s limit. That reference-machine eligibility is not established by the local timings.

6.2 Inference asset

The retained inference asset is runs/v2-local/model.npz, reported size 549482 bytes (approximately 0.524 MiB). The NumPy archive stores MLP weights, training means and scales, the vote coefficient, fallback flag and feature version. No nearest-neighbor index, answer table or cached predictions are required. The single uncompressed checkpoint is well below the 16 MiB inference-asset budget. Checkpoint size is different from peak process RAM, which was not measured for this run.

SHA256 of the retained Windows checkpoint

```
9C5C1471F021FD8506E28319794AC9455
B83FA87A85F0785246BECD8A851CBD0
```

The two lines above form one uninterrupted 64-character hash. It identifies the local checkpoint reported by the student, not the separate Linux reference checkpoint. The matching prediction entry point is submission.py, which imports predict from embedding_method.py; training is implemented in train.py. Archived experimental checkpoints are not inference assets for this retained method.

Official evidence files for the retained reproduction are runs/v2-local/validation-score-cleanup.json and runs/v2-local/test-score-recheck.json, together with their corresponding prediction JSON files. A public source revision should be linked to this exact checkpoint when the repository is published.

7 Reproduction commands

Run from code/ using Python 3.12. Preserve the supplied data, LICENSE.txt and NOTICE.txt. The retained checkpoint reproduces the score without training. An audit bank should be passed through the official runner; the method uses its bank and arrays arguments and does not assume a fixed bank size or question ID.

7.1 Environment and data checks

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python check_data.py
$env:OMP_NUM_THREADS="4"
$env:OPENBLAS_NUM_THREADS="4"
$env:MKL_NUM_THREADS="4"
```

The course requirements.txt pins numpy==2.5.3. The commands below use PowerShell continuation backticks; no spaces should follow a continuation backtick.

7.2 Reproduce the retained validation and test scores

```
python predict.py --method submission.py `
  --checkpoint runs/v2-local/model.npz --split validation `
  --output runs/v2-local/validation-predictions-reproduce.json
python score_predictions.py --split validation `
  --predictions runs/v2-local/validation-predictions-reproduce.json `
  --output runs/v2-local/validation-score-reproduce.json

python predict.py --method submission.py `
  --checkpoint runs/v2-local/model.npz --split test `
  --output runs/v2-local/test-predictions-reproduce.json
python score_predictions.py --split test `
  --predictions runs/v2-local/test-predictions-reproduce.json `
  --output runs/v2-local/test-score-reproduce.json
```

Expected counts are 736/798 on validation and 374/467 on test. Test commands here reproduce an already reported checkpoint; they are not a model-selection loop.

7.3 Reproduce training and validation analyses

```
python train.py --epochs 60 --hidden 64 --batch-size 128 `
  --lr 0.001 --weight-decay 0.001 --dropout 0.15 `
  --seed 17 --run-dir runs/v2-reproduction
python run_ablations.py --output-root runs/ablations-reproduction
python analyze_errors.py `
  --predictions runs/v2-local/validation-predictions-reproduce.json `
  --output-dir runs/error-analysis-reproduction
```

Use a fresh training directory so the retained checkpoint is not overwritten. Different NumPy/BLAS environments can introduce numerical differences in retraining; checkpoint-based reproduction should be assessed separately. The analysis reads validation references for analysis only.

7.4 Verify the asset and measure full processes

```
Get-FileHash runs/v2-local/model.npz -Algorithm SHA256
```

For end-to-end timing, run both official evaluation commands sequentially from one saved script or one enclosing command block, checking exit status and measuring elapsed time around the two processes. Avoid human pauses between start and stop. Do not cite measure_validation.py as a measurement of this checkpoint unless its checkpoint path is confirmed: the original version targets runs/v2 rather than runs/v2-local.

8 Discussion and conclusion

V2 improves on majority voting by 48 validation questions using a compact shared scorer, candidate-relative embeddings and answer agreement. The no-embeddings ablation loses 30 correct selections, supporting the value of the supplied representations in this system. The explicit prior contributes only one observed additional validation selection beyond features that already encode agreement. More training, wider contextual representations and alternative ranking objectives did not reliably improve the reported public test result.

The most useful diagnostic is separating answer disagreement from easy agreement. V2 achieves 82.54% on the 355 validation questions with at least two different valid answers, compared with 92.23% over all validation questions. Its public test result is 80.09%. Exactly-one-correct groups account for most remaining validation errors. The model can improve candidate selection without establishing that it has learned a general mathematical verifier.

Several limitations remain. The question is not separately encoded, and length, formatting or lexical cues may act as shortcuts. The main ablations use one seed and validation-selected checkpoints. There is no measured confidence calibration, formal proof verification or complete peak-RAM measurement. Random training folds may also fail to capture differences between the easier development bank and a harder evaluation bank. None of these explanations is individually established as the cause of the observed errors.

Later development used training-only fitting and nested comparisons, but occurred after public test exposure; multiple public test results were viewed before returning to V2. Those comparisons therefore cannot establish independent generalization. An instructor audit bank would provide a more informative check. The retained artifact produces 374/467 under the official scorer, with a small inference asset and short observed local runtime, subject to the course reference-machine and protocol review.

AI assistance and reuse disclosure

OpenAI ChatGPT assisted with feature and model design, implementation, debugging, experiment planning, result interpretation and report drafting. The student executed the Windows experiments and supplied their logs; additional reference runs were executed in the assistant environment. Those environments are distinguished in the report. The implementation uses NumPy and the supplied official parser, loaders and evaluator. The supplied representations are used directly, without adding pretrained weights. Responsibility for understanding and checking the submitted code and report remains with the author.

References and evidence

[1] DASE7506. MP2 Selecting Reliable LLM Reasoning. Supplied GUIDE.md, Fall 2026. Task, constraints, assessment and submission rules.

[2] DASE7506. MP2 Code and Running Instructions. Supplied README.md. Protocol dase7506-mp2-open-hard-verifier-v5. Course repository: https://github.com/xudongwu-0/DASE7506-MP2

[3] Student experiment records. runs/v2-local, runs/majority, runs/ablations and runs/error-analysis; official validation and test scorer outputs and checkpoint hash reported in Sections 3 to 7. Later experiment logs and code are retained separately in the development archives.

[4] Course data attribution. GSM8K questions and references under the retained MIT license; candidate solutions and representations prepared with Qwen3-0.6B, with revision and hashes in data/manifest.json. Preserve data/LICENSE.txt and data/NOTICE.txt when redistributing.